

August 17, 2026

TECHNISCHE UNIVERSITÄT DARMSTADT
CHAIR FOR APPLIED CRYPTOGRAPHY
Prof. Sebastian Faust



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Test Exam

Blockchain Technology Test

2026

Exercise 1: Short Questions / 13 points

(a) (5 points) There is exactly one correct answer per question, and each correctly solved question is worth 1 point.

- The function $f(n) = \frac{1}{\sqrt[5]{n+n^2}}$ is negligible. ☐ correct ☐ false
- If you have a functioning quantum computer, you can get all future mining rewards in Bitcoin. ☐ correct ☐ false
- Bitcoin transactions are faster than a classical card payments. ☐ correct ☐ false
- An automated market maker trades tokens for other tokens. ☐ correct ☐ false
- By definition, all cryptographic adversaries are computationally bounded. ☐ correct ☐ false

Solution: False, false, false, correct, false.

- $f(n) > \frac{1}{n^3}$ for large n , meaning it not smaller than any inverse polynomial.
- You could break the signature scheme, but the hash difficulty can be adjusted using the difficulty parameter, so mining rewards cannot be stolen..
- Confirmation takes 10 minutes per block, whereas card payments confirm essentially instantaneously.
- Correct.
- One might also consider computationally *unbounded* adversaries, e.g. for perfect hiding/binding.

(b) (4 points) Recall the definition of a commitment scheme (Gen, Commit, Open) from the lecture. Fill in the three blanks in the definition of the **computational binding** property below.

Binding. A commitment scheme (Gen, Commit, Open) is *computationally binding*, if for every PPT adversary $\mathcal{A}$ it holds that

$$\Pr \left[\begin{array}{c|c} m, m' \neq \perp \wedge & \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda) \\ \dots \leftarrow \mathcal{A}(pp) \\ m \leftarrow \text{Open}_{pp}(c, d) \\ m' \leftarrow \text{Open}_{pp}(c, d') \end{array} \end{array} \right] \leq \dots$$

Solution:

- Left: $m \neq m'$, i.e. $\mathcal{A}$ wins if it opens c to two different messages.
- Middle: (c, d, d') .
- Right: $\text{negl}(\lambda)$.

- (c) (4 points) Can you use the PLONK proof system introduced in the lecture with KZG commitments only, or can it be used with any polynomial commitment scheme? Decide and explain your answer.

Solution: Any polynomial commitment scheme can be used. Reason: PLONK is a *polynomial* IOP, which determines how the verifier interacts with the prover and which polynomial oracles it accesses. A polynomial commitment scheme is an instantiation of said oracle, since it allows the verifier to open positions of the polynomial.

Exercise 2: King Consensus / 17 points

Consider n parties $P_1, \dots, P_n$ connected by point-to-point authenticated channels out of which at most t are malicious (byzantine). In this exercise, we consider a weaker consensus variant called **King Consensus**. In a king consensus protocol, one party P_κ is fixed as the so-called *king* of the execution before the protocol starts, and this party is used to break ties. As in regular consensus, every party P_i holds an input p_i and produces an output o_i .

- (a) (4 points) A king consensus protocol has to fulfill the following three properties. Fill in the blank for **validity**. It is the same property as for regular consensus.

Termination: The protocol terminates after finitely many rounds.

King agreement: If the king P_κ is honest, then all honest parties decide on the same value, i.e., if P_i, P_j are honest, then $o_i = o_j$.

Validity:

.....

.....

Solution: Validity: If the input values of all honest players are identical, then all honest players decide on that value, i.e., if $p_i = z$ for all honest P_i , then $o_i = z$ for all honest players P_i . (see lecture)

- (b) (8 points) Assume you are given a king consensus protocol Π_{King} that satisfies the three properties above for a corruption threshold of t . Construct a regular **consensus** protocol from it, and argue why your construction satisfies **termination**, **agreement** and **validity**.

Solution: Construction. Idea: run Π_{King} $t + 1$ times, using a different king in every iteration:

1. Every party P_i sets $v_i^{(0)} \leftarrow p_i$.
2. For $j = 1, \dots, t + 1$:
 - (a) All parties run Π_{King} with king P_j , where P_i uses $v_i^{(j-1)}$ as its input.
 - (b) P_i sets $v_i^{(j)}$ to its output of this iteration.
3. **Output:** P_i outputs $o_i \leftarrow v_i^{(t+1)}$.

Termination. Each of the $t + 1$ iterations terminates after finitely many rounds, and $t + 1$ is finite.

Agreement. Since the adversary corrupts at most t parties, some iteration j has an honest king. By *king agreement* all honest parties hold the same value v after iteration j . They then enter iteration $j + 1$ with the same input v , so *validity* makes them output v again, and inductively v survives all remaining iterations. Hence all honest parties output v .

Validity. If all honest parties start with the same z , validity of Π_{King} preserves z in every iteration, so they all output z .

- (c) (2 points) Assume that Π_{King} terminates after R rounds. How many rounds does your consensus protocol from the previous part need? Briefly justify.

Solution: We execute Π_{King} $t + 1$ times, requiring $(t + 1) \cdot R$ rounds overall.

- (d) (3 points) Now assume additionally that $t < \frac{n}{2}$. Briefly motivate why it is also possible to build a **byzantine broadcast** protocol from king consensus.

Solution: By the previous part, king consensus gives regular consensus (byzantine agreement), and from the lecture we know that for $t < \frac{n}{2}$ byzantine agreement implies byzantine broadcast.

Exercise 3: Ethereum Smart Contracts / 18 points

Below you find two Solidity smart contracts. Each of them contains a vulnerability, meaning that the actual behavior of the contract deviates from the intended behavior, which is specified in the comment above the contract.

```
1  pragma solidity ^0.8.0;
2
3  // Contract_1_Charity collects donations. Every donor may take back the
4  // money they have donated at any time.
5
6  contract Contract_1_Charity {
7      mapping(address => uint) public donations;
8
9      function donate() external payable {
10         donations[msg.sender] += msg.value;
11     }
12
13     function withdraw() external {
14         uint amount = donations[msg.sender];
15         require(amount > 0, "Nothing to withdraw");
16
17         (bool success, ) = msg.sender.call{value: amount}("");
18         require(success, "Transfer failed");
19
20         donations[msg.sender] = 0;
21     }
22 }
23
24 // Contract_2_Lottery lets a player pay 1 ether to guess a number between
25 // 0 and 9. If the guess matches the winning number, the player receives
26 // the whole pot. Every player should win with probability exactly 1/10.
27
28 contract Contract_2_Lottery {
29     uint public lastWin;
30
31     function guess(uint8 number) external payable {
32         require(msg.value == 1 ether, "Ticket costs 1 ether");
33         require(number < 10, "Number must be between 0 and 9");
34
35         uint8 rand = uint8(uint(keccak256(abi.encodePacked(block.timestamp,
36             ↪ block.number))) % 10);
37
38         if (rand == number) {
39             payable(msg.sender).transfer(address(this).balance);
40             lastWin = block.timestamp;
41         }
42     }
43 }
```

- (a) (8 points) Select **one** of the two contracts. Name its vulnerability, and briefly explain how to exploit it.

I will analyze contract _____.

Solution:

- **Contract_1_Charity: Reentrancy.**
withdraw calls the sender *before* resetting donations[msg.sender]. The call

hands control to the caller, who can call `withdraw` again while the stored donation is still unchanged. This allows an attacker to drain the entire contract balance by repeatedly calling `withdraw`.

- **Contract_2_Lottery: Bad randomness.**

The “random” number is derived deterministically from `block.timestamp` and `block.number`, which are public and already fixed when the guess is submitted, so it is not unpredictable. An attacker can deploy a contract that computes `rand` with the same expression and only calls `guess` with the winning number. It wins with probability 1.

- (b) (10 points) For each of the two contracts, a modified version is proposed below. For the contract you have chosen in subtask a), decide if the fix eliminates the vulnerability and give an argument why.

```
1  pragma solidity ^0.8.0;
2
3  // Proposed fix for Contract_1_Charity
4
5  contract Fix_1_Charity {
6      mapping(address => uint) public donations;
7
8      function donate() external payable {
9          donations[msg.sender] += msg.value;
10     }
11
12     function withdraw() external {
13         uint amount = donations[msg.sender];
14         require(amount > 0, "Nothing to withdraw");
15
16         donations[msg.sender] = 0;
17
18         (bool success, ) = msg.sender.call{value: amount}("");
19         require(success, "Transfer failed");
20     }
21 }
22
23 // Proposed fix for Contract_2_Lottery
24
25 contract Fix_2_Lottery {
26     uint public lastWin;
27
28     function guess(uint8 number) external payable {
29         require(msg.value == 1 ether, "Ticket costs 1 ether");
30         require(number < 10, "Number must be between 0 and 9");
31
32         uint8 rand = uint8(uint(keccak256(abi.encodePacked(
33             block.timestamp,
34             block.number,
35             blockhash(block.number - 1),
36             msg.sender
37         )))) % 10);
38
39         if (rand == number) {
40             payable(msg.sender).transfer(address(this).balance);
41             lastWin = block.timestamp;
42         }
43     }
44 }
```

Solution:

Fix_1_Charity: Yes, this fixes the vulnerability. The state is now updated *before* the external call. When the attacker re-enters `withdraw`, `amount == 0`.

Fix_2_Lottery: No, this does *not* fix the vulnerability. The modification only mixes *more public* values into the hash. All four inputs are known to the caller: `blockhash(block.number - 1)` is the hash of the previous block, and `msg.sender` is the attacker's own address.

Exercise 4: A Sum Test for PLONK

/ 15 points

Recall that the PLONK toolbox introduced in the lecture includes a *product test*. In this task, your goal is to construct and investigate the related sum test protocol. Let ω be a primitive k -th root of unity in $\mathbb{F}_p$, let $\Omega = \{\omega^i \mid 0 \leq i < k\}$ be the multiplicative subgroup generated by ω , and let $Z_\Omega(X) = X^k - 1$ be the vanishing polynomial on Ω .

The **product test** proves that $\prod_{x \in \Omega} f(x) = 1$. Its central idea is to let the prover construct an auxiliary polynomial $t \in \mathbb{F}_p^{(\leq d)}[X]$ with

$$t(1) = f(1), \quad t(\omega^s) = \prod_{i=0}^s f(\omega^i) \quad \text{for } s = 1, \dots, k-1,$$

so that $t(\omega \cdot x) = t(x) \cdot f(\omega \cdot x)$ holds for all $x \in \Omega$. Then one can prove the following lemma:

Lemma: If

- (1) $t(\omega^{k-1}) = 1$ and
- (2) $t_{diff}(x) = t(\omega \cdot x) - t(x) \cdot f(\omega \cdot x) = 0$,

for all $x \in \Omega$, then it holds that $\prod_{x \in \Omega} f(x) = 1$.

Condition (2) can be proven with a **zero test**, in which the prover sends the quotient $q(X) \leftarrow \frac{t_{diff}(X)}{Z_\Omega(X)}$, which exists if and only if t_{diff} vanishes on all of Ω .

(a) (8 points) We now want to build a **sum test** instead, to prove for $f \in \mathbb{F}_p^{\leq d}[X]$ that

$$\sum_{x \in \Omega} f(x) = 0.$$

Fill in the blanks in the protocol description of the sum test below. The sum test also employs an auxiliary polynomial t that is defined differently as for the product test. Commitments to functions are marked with a blue background.

Prover $P(f)$

Verifier $V(\textcolor{blue}{f})$

Construct t and t_{diff} , where:

$$t_{diff}(X) = t(\omega X) - \dots\dots\dots$$

$$q(X) \leftarrow \frac{t_{diff}(X)}{Z_\Omega(X)}$$

$$\textcolor{blue}{t}, \textcolor{blue}{q} \in \mathbb{F}_p^{(\leq d)}[X]$$

$$r \xleftarrow{\$} \mathbb{F}_p$$

r

Query t at $\omega^{k-1}, r, \omega r$

Query q at r and f at ωr

Accept if

$$t(\omega^{k-1}) \stackrel{?}{=} \dots\dots\dots$$

$$\dots\dots\dots \stackrel{?}{=} q(r) \cdot Z_\Omega(r)$$

Solution: Here, auxiliary polynomial t is the running *sum*, i.e. $t(1) = f(1)$ and $t(\omega^s) = \sum_{i=0}^s f(\omega^i)$. The product is replaced by a sum everywhere:

1. $(t(X) + f(\omega X))$, so that $t_{diff}(X) = t(\omega X) - (t(X) + f(\omega X))$.
2. 0.
3. $t(\omega r) - t(r) - f(\omega r)$.

- (b) (4 points) Assume we instantiate this protocol with the KZG polynomial commitment scheme, meaning that we send commitments to polynomials and polynomial evaluation proofs instead of the polynomials themselves. Is the resulting protocol succinct? Justify your answer.

Solution: Yes. KZG commitments and their opening proofs have constant size (independent of the polynomial degree). $Z_{\Omega}(r) = r^k - 1$ can be computed in time $O(\log k)$ (square-and-multiply). Proof size and verifier runtime are therefore at most logarithmic in the size of the statement.

- (c) (3 points) Is there a way to make the protocol non-interactive? If so, name the technique and give a high-level explanation of how it works.

Solution: Yes, using the **Fiat-Shamir transform**. The protocol is *public coin*: the verifier's only message the uniformly random r . Such a challenge can be replaced by a hash of the statement and all previous messages, if the hash function H is modelled as a random oracle.

Exercise 5: Scalability and Payment Channels / 12 points

Recall the unidirectional payment channel from the lecture. Alice (with key pair (pk_A, sk_A)) funds a channel towards Bob (with key pair (pk_B, sk_B)) with 1 BTC. The channel consists of an on-chain *funding* transaction, an off-chain *refund* transaction and a sequence of off-chain *update* transactions $\text{Update}_1, \text{Update}_2, \dots$ that shift more and more money to Bob.

- (a) (5 points) Alice and Bob set up their channel with the following funding and refund transactions.

Transaction Funding	
Input Tx:	$\mathcal{H}(T_0)$
Out script(σ_A):	$\text{P2PKH}(pk_A, \sigma_A)$
Value:	1 BTC
Witness:	$\text{Sign}_{sk_A}([\text{Funding}])$

Transaction Refund	
Input Tx:	$\mathcal{H}(\text{Funding})$
Out script(σ):	$\text{P2PKH}(pk_A, \sigma)$
Value:	1 BTC
Timelock:	$t = 30 \text{ days}$
Witness:	$\text{Sign}_{sk_A}([\text{Refund}])$

The update transactions are as in the lecture.

Describe an attack by which Alice can trade with Bob (using update transactions) and still get all of her funds back.

Solution: Alice can perform an overwriting attack, since she is not bound to publishing the refund transaction shown to Bob. She might publish a new refund transaction without a timelock, and refund her money immediately, even though she has already traded Bob for parts of the funds. This can be avoided by requiring an additional signature by Bob to spend the funding transaction.

- (b) (5 points) Alice and Bob repair the transactions from the previous part to prevent the attack. Consider the following example of an update transaction:

Transaction Update _i		
Input Tx:	$\mathcal{H}(\text{Funding})$	
Out script(σ):	$\text{P2PKH}(pk_A, \sigma)$	$\text{P2PKH}(pk_B, \sigma)$
Value:	0.10 BTC	0.90 BTC
Timelock:	$t = 31$ days	
Witness:	$\text{Sign}_{sk_A}([\text{Update}_i]), ?$	

Which security flaw does this update transaction contain?

Solution: The update becomes valid *later* than the refund (31 vs. 30 days), instead of strictly earlier.

Alice can use the channel until Bob holds Update_i giving him 0.90 BTC. On day 30 the refund becomes valid and she publishes it, taking the full 1 BTC back. Bob's update only becomes valid on day 31, but by then the funding output is already spent, so he loses the 0.90 BTC.

- (c) (2 points) Payment channels are not the only way to improve the scalability of a blockchain. Name one alternative approach and briefly state how it works.

Solution: For instance

- **Rollups** (optimistic or zk-rollups): a coordinator processes many off-chain transactions, and posts only the new state root plus the transaction data to L1.

Exercise 6: Anonymity

/ 12 points

Bob wants to anonymously send money, so he uses the ZeroCoin protocol from the lecture. Bob's basecoin address is addr_B . Below you see all ZeroCoin *Mint* and *Spend* transactions that were recorded in the relevant period in which Bob minted and spent his coin.

Mint transactions						
Tx	Block	Sender	Commitment	Amount	Fee	Size
M_1	959 412	addr_C	0x9f4a...	1.00 BTC	0.0002 BTC	372 B
M_2	959 414	addr_D	0x2c7e...	0.50 BTC	0.0005 BTC	368 B
M_3	959 415	addr_N	0x6b21...	3.75 BTC	0.0001 BTC	374 B
M_4	959 418	addr_E	0x5d33...	1.00 BTC	0.0001 BTC	371 B
M_5	959 420	addr_B	0xb108...	2.50 BTC	0.0005 BTC	369 B
M_6	959 421	addr_F	0xa761...	0.50 BTC	0.0002 BTC	373 B
M_7	959 423	addr_O	0x3c05...	0.25 BTC	0.0003 BTC	370 B
M_8	959 426	addr_G	0x0e9c...	1.00 BTC	0.0005 BTC	372 B
M_9	959 428	addr_P	0x8f7d...	0.10 BTC	0.0002 BTC	368 B
M_{10}	959 430	addr_Q	0x14ab...	5.00 BTC	0.0001 BTC	374 B
M_{11}	959 432	addr_R	0xe530...	0.25 BTC	0.0003 BTC	371 B
M_{12}	959 435	addr_S	0x7a92...	1.00 BTC	0.0002 BTC	369 B

Spend transactions						
Tx	Block	Serial Number	Amount	Recipient	Fee	Size
S_1	959 473	0x71bd...	1.00 BTC	addr_H	0.0210 BTC	45.9 KB
S_2	959 475	0x3ea0...	0.50 BTC	addr_I	0.0185 BTC	46.2 KB
S_3	959 478	0xc45f...	0.25 BTC	addr_J	0.0230 BTC	45.8 KB
S_4	959 480	0x8a26...	3.75 BTC	addr_K	0.0195 BTC	46.0 KB
S_5	959 482	0xd902...	1.00 BTC	addr_L	0.0185 BTC	45.7 KB
S_6	959 484	0x1f68...	0.05 BTC	addr_M	0.0230 BTC	46.3 KB
S_7	959 487	0x4d13...	0.50 BTC	addr_T	0.0210 BTC	45.9 KB
S_8	959 489	0xb87c...	2.50 BTC	addr_U	0.0230 BTC	46.1 KB
S_9	959 491	0x259e...	3.75 BTC	addr_V	0.0195 BTC	45.8 KB
S_{10}	959 494	0xf341...	7.20 BTC	addr_W	0.0210 BTC	46.0 KB
S_{11}	959 496	0x6cea...	0.50 BTC	addr_X	0.0185 BTC	46.2 KB
S_{12}	959 499	0x0ab7...	1.00 BTC	addr_Y	0.0195 BTC	45.7 KB

(a) (4 points) Who did Bob send his money to? Justify your answer.

Solution: Bob paid the owner of addr_U .

A ZeroCoin is always spent for the amount it was minted with. Bob mints a coin of 2.50 BTC. S_8 is the only spend of 2.50 BTC, hence it must open Bob's commitment. His anonymity set has size 1.

- (b) (3 points) Given the situation above, explain why ZeroCoin nevertheless offers stronger anonymity guarantees than Bitcoin.

Solution: ZeroCoin spend transactions are at least anonymous within the anonymity set related to the transaction value. This is not the case for Bitcoin, where you can directly trace all transactions.

- (c) (3 points) How could a **centralized** mixer help Bob hide the recipient of his payment? Briefly describe how such a mixer works.

Solution: Bob sends his coins to a mixing service, which collects coins from many users, holds them for a while, and then pays out the corresponding amounts from its pool to fresh output addresses. Since the payouts come from a common pool and are done in a random order, an observer cannot link payments.

- (d) (2 points) Name one advantage of **decentralized** mixers over centralized mixers.

Solution: For instance:

- **Lower risk of theft:** no one can run away with your money.
- **No trusted party for privacy:** the centralized mixer knows exactly who paid whom.
- **No single point of failure:** self explanatory.

Exercise 7: Automated Market Makers / 11 points

Consider an automated market maker (AMM) holding a pool of two tokens A and B . Initially, the pool contains 50 tokens A and 200 tokens B . We ignore trading fees in this exercise.

- (a) (2 points) What is the automated market maker's constant k ?

Solution: $k = 50 \cdot 200 = 10000$.

- (b) (3 points) A trader buys 10 tokens A from the pool. How many tokens B do they have to put up?

Solution: It must hold that:

$$(50 - 10) \cdot (200 + x) = k = 10000 \quad \Rightarrow \quad x = 50.$$

- (c) (3 points) Briefly explain what arbitrage for automated market makers is. If someone is profiting from an arbitrage opportunity, who is paying for it?

Solution: An AMM prices the two tokens purely by the ratio of its own reserves and has no idea what they trade for elsewhere. If the pool price and an external market price drift apart, this creates a risk-free profit opportunity, called *arbitrage*. While the arbitrageur profits from the trade, the AMM's *liquidity providers*, so the people who originally deposited money in the AMM, pay the difference.

- (d) (3 points) Assume that the external market price is $1 A = 6.25 B$. Is there an arbitrage opportunity after the trade in the second part? If so, how many tokens A or B can you make from exploiting it?

Solution: No, there is no arbitrage opportunity. The implied price of 1 token A in the pool is

$$\frac{\#B}{\#A} = \frac{250}{40} = 6.25,$$

which is exactly the external market price.